

Client Portal Engineer (Frontend & Full-Stack) — Take-Home Assessment

Role: Frontend & Full-Stack Engineer — owns Client Portal development, Scale R&D, Rapyd, Dubai **Time budget:** ~4–5 hours total, spread over up to 5 days **Format:** Async take-home, followed by a 60-minute live review

Before you start

This exercise mirrors how the role actually works: you own a customer-facing surface end-to-end — the UX decisions, the frontend, the API behind it, and explaining what you built to a non-technical stakeholder. There's no separate team that “does the backend” or “does the design” — that's why this is a full-stack role, not a frontend-only one.

You're welcome and encouraged to use AI tools (Claude, Copilot, ChatGPT, or anything else) anywhere in this exercise, the same way you would on the job. Just note briefly, in each deliverable, where and how you used them. We're evaluating your judgment, not whether you typed every keystroke yourself.

The scenario

Merchants using our payments platform currently have no visibility into their own settlement reconciliation status. When a payout looks short, or a transaction seems to be missing, their only option is to email support — who then has to manually dig through internal tooling to find an answer. This creates support load and, worse, it makes merchants nervous about whether we're tracking their money correctly.

Your job: build the first version of a **self-serve Client Portal** where a merchant can log in and see their own settlement reconciliation status — what's matched, what's flagged, and what it means — without picking up the phone.

You've been given the same two data sources our internal reconciliation tooling uses:

- `settlement_export.csv` — the payment processor's settlement file
- `ledger_export.csv` — our internal ledger

Both files span multiple merchants. **For this exercise, build for one logged-in merchant: M-104.** Assume authentication is already solved elsewhere — you can hardcode or mock the session — but your API should be scoped by merchant as if this pattern will be reused for real auth later.

Part 1 — UX & product framing (45–60 min)

Before writing any code, write a short doc (1–2 pages) covering:

- **The user, in one sentence** — who's logging in, and what are they trying to find out?
- **Core journeys** — the 2–3 things a merchant needs to be able to do in this first version.
- **States you're designing for** — loading, empty ("you're all clear," which is a real, good outcome — don't treat it as an afterthought), error, and the populated state.
- **A rough IA / component sketch** — a wireframe, an ASCII sketch, or just a clear textual description of the screen(s) and their pieces. No code yet.
- **Definition of done** — one concrete UX commitment (e.g. an accessibility bar you're designing to) and one business metric this should move (e.g. reduction in "where's my money" tickets).

Part 2 — Build (2.5–3 hrs)

Build a small full-stack portal:

Backend (Node.js / TypeScript), scoped to merchant M-104: 1. An endpoint returning a summary (counts and dollar impact by exception reason) 2. A paginated, filterable, sortable endpoint for the merchant's individual exceptions 3. An export endpoint (CSV or JSON download) for that merchant's exceptions

Frontend (React / TypeScript, Redux or an equivalent if the state genuinely warrants it): 1. A dashboard view — summary at a glance 2. An exceptions table — paginated, sortable, filterable by reason and date 3. A detail view for a single exception (drawer, modal, or separate view — your call) 4. All four states from Part 1 actually implemented, not just the happy path 5. An export button wired to your backend endpoint 6. At least some explicit accessibility care (semantic markup, keyboard-navigable table and filters, don't rely on color alone to signal status) and one note on how you'd handle this responsively on a smaller screen

Tests: Jest + React Testing Library covering your exceptions table (filtering, sorting, empty state, error state at minimum), plus at least one test on the backend.

We care far more about the states, the accessibility basics, and whether the API shape actually serves the frontend well than about how visually polished this looks in a few hours.

Part 3 — AI/LLM add-on (30 min)

Add a feature where a merchant can click an exception and get a plain-English explanation of why it was flagged (using the exception's data as context). A mocked LLM call is completely fine.

In a short written note, tell us: - Your prompt design decisions - What the merchant sees if the call fails or returns something off-base - Whether you'd ship this to real merchants without a human review step, and why

Part 4 — Stakeholder memo (20–30 min)

Write a one-page memo (or a short recorded walkthrough, 3 minutes max) to the Head of Merchant Support — non-technical, and currently fielding these tickets manually. Cover:

- What merchants can now do themselves vs. what still needs a support ticket
- One trade-off you made under the time constraints, and what you'd change with more time

- The one metric you'd track after launch to know whether this actually reduced support load

What to submit

- Your UX/product doc (Part 1)
- A repo or zip of your code (Parts 2–3) with a short README on how to run it
- Your memo (Part 4)

What happens next

We'll schedule a 60-minute conversation where you walk us through your design decisions and your code, and we pressure-test a few of them live.